

```

}
}

int my_init(){
display("Hello Open World\n");
}

```

Let's run through this code snippet. First, we declared a volatile variable pointer, and assigned the address of the serial port (UART0). The function “*my_init()*”, is the main routine. It merely calls the function “*display()*”, which writes a string to the UART0.

Engineers familiar with base-level micro-controller programming will find this very easy. If you are not experienced in embedded systems programming, then you can stick to the basics of digital electronics. The microprocessor is an integrated chip, with input/output lines, different ports, etc. The ARM926EJ-S has four serial ports (information obtained from its data-sheet); and they have their data lines (the address). When the processor is programmed to write data to one of the serial ports, it writes data to these lines. That's what this program does.

The next step is to develop the start-up code for the processor. When a processor is powered on, it jumps to a specified location, reads code from that location, and executes it. Even in the case of a reset (like on a desktop machine), the processor jumps to a predefined location. Here's the start-up code, *startup.s* :

```

.global _Start
_Start:
LDR sp, = sp_top
BL my_init
B .

```

In the first line, *_Start* is declared as global. The next line is the beginning of *_Start*'s code. We set the address of the stack to *sp_top*. (The instruction *LDR* will move the data value of *sp_top* in the stack pointer (*sp*). The instruction *BL* will instruct the processor to jump to *my_init* (previously defined in *init.c*). Then the processor will step into an infinite loop with the instruction ‘*B .*’, which is like a *while(1)* or *for(;;)* loop. If we don't do this, our system will crash. The basics of embedded systems programming is that our code should run into an infinite loop.

Now, the final task is to write a linker script for these two files (*linker.ld*):

```

ENTRY(_Start)
SECTIONS
{
    . = 0x10000;
    startup : { startup.o(.text)}
    .data : {*(.data)}
    .bss : {*(.bss)}
}

```

```

. = . + 0x500;
sp_top = .;
}

```

The first line tells the linker that the entry point is *_Start* (defined in *startup.s*). As this is a basic program, we can ignore the *Interrupts* section. The QEMU emulator, when executed with the *-kernel* option, starts execution from the address 0x10000, so we must place our code at this address. That's what we have done in Line 4. The section ‘*SECTIONS*’, defines the different sections of a program. In this, *startup.o* forms the text (code) part. Then comes the subsequent data and the *bss* part. The final step is to define the address of the stack pointer. The stack usually grows downward, so it's better to give it a safe address. We have a very small code snippet, and can place the stack at 0x500 ahead of the current position. The variable “*sp_top*” will store the address for the stack.

We are now done with the coding part. Let's compile and link these files. Assemble the *startup.s* file with:

```
$ arm-none-eabi-as -mcpu=arm926ej-s startup.s -o startup.o
```

Compile *init.c*:

```
$ arm-none-eabi-gcc -c -mcpu=arm926ej-s init.c -o init.o
```

Link the object files into an ELF file:

```
$ arm-none-eabi-ld -T linker.ld init.o startup.o -o output.elf
```

Finally, create a binary file from the ELF file:

```
$ arm-none-eabi-objcopy -O binary output.elf output.bin
```

The above instructions are easy to understand. All the tools used are part of the ARM tool-chain. Check their *help/man* pages for details.

After all these steps, finally we will run our program on the QEMU emulator:

```
$ qemu-system-arm -M versatilepb -nographic -kernel output.bin
```

The above command has been explained in previous articles, so we won't go into the details. The binary file is executed on QEMU and will write the message ‘Hello Open World’ to UART0 of the ARM926EJ-S, which QEMU redirects as output in the terminal. 

By: Manoj Kumar

The author is a freelance developer and trainer. He leads a team in Linux kernel programming, Linux administration, cluster computing, embedded systems and QT/GTK programming on Linux. You can reach him at manoj@nixitsolutions.com and read his blog, www.open4india.blogspot.com, to share your Linux knowledge.